

MSSV: 23127195

Họ và tên : Trần Mạnh Hùng

Câu 1:

Dưới đây là 6 yêu cầu có ý nghĩa kiến trúc (ASRs) rút ra từ tình huống UniFlow, được phân tích chi tiết:

1. Khả năng hoạt động ngoại tuyến (Offline-first) khi điểm danh

- **Phân loại:** Ràng buộc phi chức năng (Tính sẵn sàng / Độ tin cậy) gắn với yêu cầu chức năng (Điểm danh).
- **Vì sao ảnh hưởng kiến trúc (chứ không phải chi tiết code):** ta không thể giải quyết việc này bằng cách viết một khối try-catch kiểm tra mạng trong code UI.
 - Về mặt kiến trúc, nó buộc phía Client (Mobile App) phải có một lớp lưu trữ cục bộ (Local Database) và một hàng đợi tác vụ (Background Task Queue).
 - Phía Backend phải thiết kế một cơ chế Đồng bộ hóa (Sync Mechanism) và Giải quyết xung đột dữ liệu (Conflict Resolution) khi mạng có lại, thay vì mô hình Client-Server gọi API đồng bộ (Synchronous) thông thường.

2. Tích hợp hệ thống cũ (Legacy System) qua file CSV ban đêm

- **Phân loại:** Yêu cầu chức năng (Tích hợp / Giao tiếp hệ thống) và Ràng buộc môi trường.
- **Vì sao ảnh hưởng kiến trúc:** Hệ thống không thể sử dụng phong cách *Call and Return* (như REST API) để nói chuyện với hệ thống 2018.
 - Kiến trúc bắt buộc phải có một khối **Batch Processing** (theo phong cách *Batch Sequential* mà thầy Khoa có nhắc trong slide). Ta sẽ phải thiết kế các thành phần như: Scheduler (lập lịch), ETL Pipeline (Extract-Transform-Load) để đọc CSV, biến đổi dữ liệu, và cập nhật vào Database mới mà không gây lock dữ liệu của người dùng vào ban ngày.

3. Chịu tải 5.000 người dùng đồng thời (CCU) khi đăng ký môn

- **Phân loại:** Yêu cầu phi chức năng (Hiệu năng / Khả năng mở rộng - Scalability).
- **Vì sao ảnh hưởng kiến trúc:** Ta không thể giải quyết 5.000 CCU chỉ bằng cách tối ưu thuật toán (như dùng HashMap thay vì Array).

- Con số này quyết định **Topology** (sơ đồ triển khai) của hệ thống: Phải có Load Balancer (Bộ cân bằng tải) đứng trước, ứng dụng phải thiết kế ở dạng Stateless (không lưu trạng thái ở RAM server) để có thể nhân bản (scale out) ra nhiều server vật lý. Đồng thời, kiến trúc cơ sở dữ liệu phải có tầng Cache (như Redis) để giảm tải truy vấn đọc.

4. Gửi thông báo dựa trên các sự kiện (đổi lịch, nộp bài, có điểm)

- **Phân loại:** Yêu cầu chức năng.
- **Vì sao ảnh hưởng kiến trúc:** Nếu lập trình viên chỉ đơn thuần viết hàm taoThongBao() và gọi nó (Call and Return) ở cuối hàm nhapDiem(), hệ thống sẽ bị dính (tight coupling) và chậm .
 - Kiến trúc sư phải quyết định sử dụng phong cách **Event-driven / Implicit Invocation**. Cần phải đưa vào một thành phần Message Broker (ví dụ: RabbitMQ, Kafka hoặc Redis Pub/Sub) làm trung tâm. Các module như Quản lý điểm, Quản lý lịch học chỉ việc phát (emit) sự kiện, và module Thông báo sẽ lắng nghe (subscribe) để xử lý ngầm (asynchronous).

5. Giới hạn nguồn lực và thời gian (1 SA, 4 Dev, 1 DevOps, 6 tháng)

- **Phân loại:** Ràng buộc dự án / Yêu cầu phi chức năng (Tính khả thi bảo trì).
- **Vì sao ảnh hưởng kiến trúc:** Đây là trở ngại nguồn lực thực tế nhất. Với cấu trúc team siêu nhỏ này, kiến trúc sư **tuyệt đối không được** chọn phong cách Microservices phân tán (vì chi phí vận hành, tracing, setup CI/CD sẽ giết chết tiến độ dự án).
 - Thay vào đó, nó định hướng kiến trúc tổng thể phải là **Modular Monolith** kết hợp với **Layered Architecture**. Toàn bộ team sẽ làm việc trên một codebase duy nhất, phân tách rõ ràng bằng các layer (Presentation, Business Logic, Data Access) để dev dễ code, dễ test và DevOps dễ deploy.

6. Bảo mật Zero-trust và phân quyền chặt chẽ (RBAC)

- **Phân loại:** Yêu cầu phi chức năng (Bảo mật - Security).
- **Vì sao ảnh hưởng kiến trúc:** Zero-trust nghĩa là không tin tưởng bất kỳ ai, kể cả request đến từ mạng nội bộ của trường.
 - Về kiến trúc, bạn không thể rải rác các đoạn code kiểm tra if (user.role == 'admin') ở hàng trăm API khác nhau. Kiến trúc bắt buộc phải có một cổng bảo vệ trung tâm — thường là một **API Gateway** hoặc một lớp Middleware bảo mật chuyên biệt đứng trước toàn bộ hệ thống để xác thực

(Authentication) bằng Token (như JWT) và phân quyền (Authorization) trước khi request chạm được vào Business Logic Layer.

Câu 2:

ĐỀ XUẤT KIẾN TRÚC TỔNG THỂ CHO UNIFLOW

Kiến trúc đề xuất là **Kiến trúc Lai (Hybrid Architecture)**, kết hợp 4 phong cách: Client-Server, Layered, Event-driven và Batch Sequential.

1. Khối giao tiếp Frontend - Backend: Chọn Client-Server

- **Giải thích:** Tách biệt rõ ràng ứng dụng người dùng (Client) và máy chủ xử lý trung tâm (Server).
- **Gắn với 2 ràng buộc:**
 1. **Mạng Wi-Fi chấp chờn:** Client (Mobile app) có thể hoạt động offline, lưu tạm dữ liệu điểm danh và gửi request lên Server đồng bộ khi có mạng.
 2. **Bảo mật Zero-trust:** Server đóng vai trò là chốt chặn duy nhất, kiểm duyệt mọi token và quyền (RBAC) trước khi cho phép đụng vào dữ liệu.
- **Phương án bị loại: Peer-to-Peer (P2P).** Lý do loại: P2P phân tán dữ liệu và quyền điều khiển ở các node, cực kỳ khó kiểm soát bảo mật tập trung và không phù hợp với môi trường quản lý học thuật cần tính toàn vẹn dữ liệu tuyệt đối (điểm số, lịch thi).

2. Khối xử lý nghiệp vụ lõi (Backend): Chọn Layered Architecture (Modular Monolith)

- **Giải thích:** Tổ chức mã nguồn backend thành các lớp (Presentation -> Business Logic -> Data Access) bên trong một khối ứng dụng duy nhất.
- **Gắn với 2 ràng buộc:**
 1. **Nguồn lực mỏng (1 SA, 4 Dev, 1 DevOps, 6 tháng):** Code chung một khối chia lớp giúp team nhỏ dễ quản lý mã nguồn, dễ test và DevOps chỉ cần deploy một artifact duy nhất.
 2. **5.000 CCU:** Kiến trúc Monolith nếu thiết kế lớp Data Access tốt (kết hợp Redis Cache) và đặt sau Load Balancer thì việc nhân bản (scale out) và server vật lý để gánh 5.000 user là hoàn toàn khả thi và tiết kiệm chi phí setup.
- **Phương án bị loại: Microservices.**

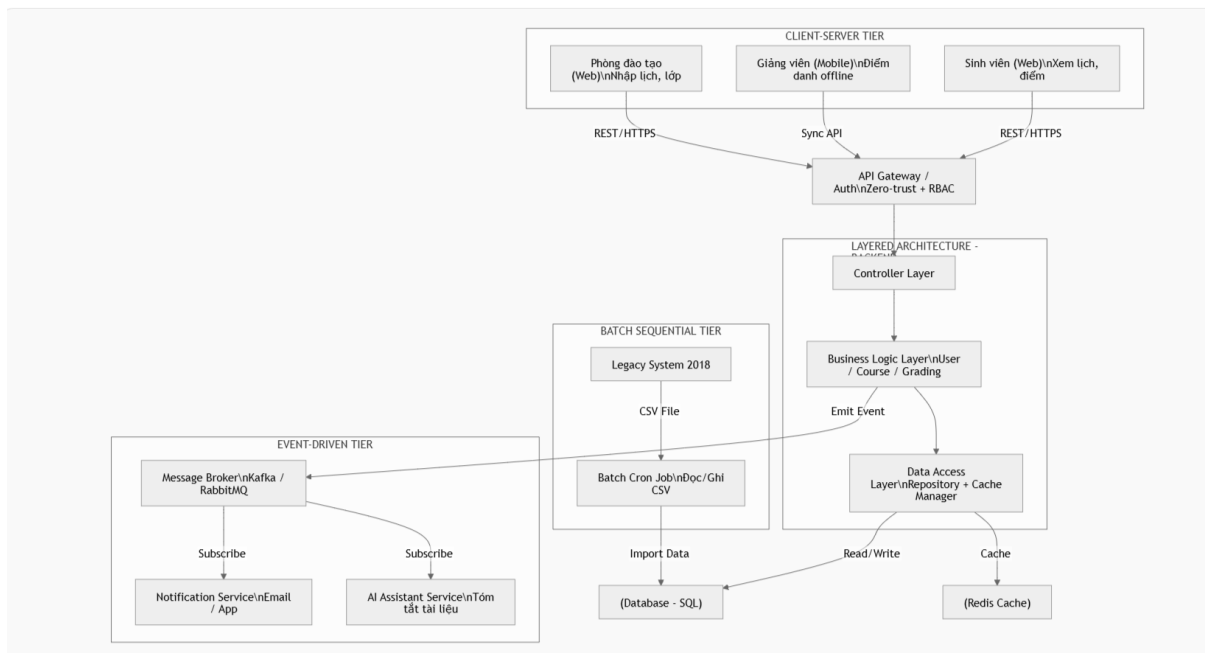
- *Lý do loại:* Overhead (chi phí) để setup hạ tầng phân tán, CI/CD, tracing cho Microservices sẽ vắt kiệt sức của 1 DevOps và 4 Dev, chắc chắn làm vỡ tiến độ 6 tháng.

3. Khối xử lý bất đồng bộ: Chọn Event-driven (Implicit Invocation)

- **Giải thích:** Các thành phần không gọi nhau trực tiếp mà phát ra (emit) các sự kiện vào một Message Broker để các dịch vụ khác tự lắng nghe và xử lý ngầm.
- **Gắn với 2 ràng buộc:**
 1. **Chức năng thông báo sự kiện:** Cần gửi thông báo khi "đổi lịch", "nộp bài", "có điểm" mà không làm gián đoạn luồng công việc chính của người dùng.
 2. **Trợ giảng AI:** Việc AI đọc và tóm tắt tài liệu rất tốn thời gian xử lý. Phải đẩy vào luồng chạy ngầm để không làm treo hệ thống.
- **Phương án bị loại:** *Call and Return (Gọi hàm đồng bộ trực tiếp).*
 - *Lý do loại:* Nếu giảng viên bấm "Lưu điểm", hệ thống gọi hàm gửi email đồng bộ cho 100 sinh viên, request sẽ bị treo rất lâu. Dưới tải 5.000 CCU, server sẽ cạn kiệt thread và sập.

4. Khối tích hợp dữ liệu cũ: Chọn Batch Sequential

- **Giải thích:** Xử lý dữ liệu theo từng lô lớn, xong bước này mới qua bước khác, không cần tương tác của người dùng.
- **Gắn với 2 ràng buộc:**
 1. **Hệ thống 2018:** Chỉ hỗ trợ xuất/nhập file CSV, không có API.
 2. **Chỉ chạy ban đêm:** Phù hợp với đặc thù gom dữ liệu thành một "lô" (batch) và chạy định kỳ lúc hệ thống ít tải nhất.
- **Phương án bị loại:** *Pipe-and-Filter.*
 - *Lý do loại:* Dù cùng là Data Flow, nhưng Pipe-and-Filter yêu cầu dữ liệu chảy liên tục xuyên suốt các bộ lọc. Ở đây ta bị ràng buộc phải đợi gom xong 1 cục file CSV từ hệ thống cũ vào ban đêm mới bắt đầu chạy, nên Batch Sequential (chờ xong từng lô) là lựa chọn chính xác.



Câu 3:

Lựa chọn nền tảng chính: Phương án X (Kiến trúc Layered Client-Server tập trung)

Mặc dù ở Câu 2 chúng ta có lai thêm Event-driven để xử lý thông báo, nhưng **nền tảng cốt lõi (Core Foundation)** bắt buộc phải là hệ thống tập trung (Modular Monolith / Layered). Việc tách thành "nhiều dịch vụ độc lập hơn" (Phương án Y - mang hơi hướng Microservices/SOA) trong hoàn cảnh này là một cái bẫy dự án cực kỳ nguy hiểm.

3 Lý do ủng hộ Phương án X

1. **Phù hợp tuyệt đối với ràng buộc nguồn lực (1 DevOps, 4 Dev, 6 tháng):** Tách nhiều dịch vụ (Phương án Y) đồng nghĩa với việc ông DevOps duy nhất phải setup hàng loạt pipeline CI/CD, cấu hình mạng nội bộ, cơ chế giám sát (tracing) cho từng service. 4 Dev sẽ mất thời gian loay hoay xử lý lỗi gọi chéo giữa các service (network latency, timeout) thay vì tập trung code nghiệp vụ. Phương án X gom về một codebase (chia Layer rõ ràng) giúp team triển khai cực nhanh, dễ debug và chắc chắn kịp deadline 6 tháng.
2. **Đảm bảo tính nhất quán dữ liệu (Data Consistency) và Bảo mật dễ dàng hơn:** Yêu cầu bảo mật Zero-trust và tính chính xác của điểm số, lịch thi đòi hỏi quản lý giao dịch khắt khe. Trong kiến trúc tập trung, việc quản lý Transaction (ACID) của database và phân quyền (RBAC) diễn ra ở một nơi, cực kỳ an toàn. Nếu dùng Phương án Y, bạn phải xử lý bài toán "Giao dịch phân tán" (Distributed

Transactions) và truyền Token bảo mật qua lại giữa các dịch vụ, rủi ro lộ lọt và sai lệch dữ liệu là rất cao.

3. **Mức tải 5.000 CCU chưa chạm ngưỡng phải chia hệ thống:** Nhiều người lầm tưởng cứ tải cao là phải dùng Microservices. Thực tế, 5.000 CCU hoàn toàn có thể được xử lý mượt mà bởi một khối Monolith (Phương án X) nếu ta thiết kế các Layer tốt, ứng dụng phi trạng thái (Stateless) và đặt sau một Load Balancer (chạy nhiều instance song song), kết hợp với Redis Cache. Tách dịch vụ lúc này là "giết gà dùng dao mổ trâu".

2 Rủi ro / Chi phí phải chấp nhận khi chọn Phương án X

1. **Rủi ro (Single Point of Failure ở mức ứng dụng):** Vì tất cả chạy chung trong một khối ứng dụng, nếu một đoạn code ở module phụ (ví dụ: hàm đọc file CSV ban đêm hoặc hàm xuất file PDF điểm số) bị lỗi gây tràn bộ nhớ (Memory Leak), nó sẽ làm sập toàn bộ ứng dụng. Lúc đó, sinh viên cũng không thể vào xem lịch, giảng viên cũng không thể điểm danh.
2. **Nghẽn tài nguyên cục bộ do module Trợ giảng AI:** Đề bài có yêu cầu AI tóm tắt tài liệu môn học. Đây là tác vụ tốn rất nhiều CPU và RAM. Nếu AI chạy chung trong cùng một server với các API cốt lõi (như đăng ký môn, điểm danh), mỗi khi có người upload tài liệu nặng, CPU bị chiếm dụng sẽ làm chậm toàn bộ các chức năng khác của hệ thống. (Chi phí phải trả là phải mua server cấu hình rất mạnh từ đầu để trừ hao).

1 Điều kiện thay đổi quyết định (Switch sang Phương án Y)

Ta sẽ lập tức vứt bỏ Phương án X và chuyển sang Phương án Y (Tách dịch vụ độc lập)

NẾU quy mô và định hướng của module Trợ giảng AI thay đổi.

Cụ thể: Nếu nhà trường quyết định biến Trợ giảng AI thành một tính năng "át chủ bài", yêu cầu xử lý hàng ngàn tài liệu lớn liên tục, chạy các mô hình Machine Learning nặng (cần dùng server có GPU chuyên dụng) VÀ nhà trường cấp thêm ngân sách/thời gian để thuê đội ngũ AI/DevOps riêng.

- **Lý do đổi:** Lúc này, việc giữ khối AI nằm chung với khối Quản lý điểm/lịch học là bất khả thi về mặt hạ tầng vật lý (Web server cần CPU/RAM tiêu chuẩn, trong khi AI cần GPU). Bắt buộc phải tách hệ thống ra thành các dịch vụ độc lập để "cô lập" mức tiêu thụ tài nguyên và cho phép hai team phát triển ở hai tốc độ khác nhau.

Câu 4 :

1. Phân tích Nhu cầu A: Tóm tắt tài liệu khi giảng viên Upload

Quyết định: Nhu cầu A phù hợp tuyệt đối với phong cách **Pipe-and-Filter (Đường ống và Bộ lọc)**.

- **Giải thích vì sao chọn:** Quá trình tóm tắt văn bản là một chuỗi các phép biến đổi dữ liệu liên tục. Dữ liệu đầu vào (PDF) sẽ "chảy" qua từng bộ lọc (Filter) độc lập: Tách chữ $\rightarrow$ Làm sạch $\rightarrow$ Chia đoạn $\rightarrow$ Đưa cho AI $\rightarrow$ Lưu kết quả. Mỗi bộ lọc chỉ làm đúng một việc, nhận input từ đường ống (Pipe) trước và đẩy output ra đường ống sau.
- **Gắn với 2 ràng buộc cụ thể:**
 1. **Ràng buộc chức năng Trợ giảng AI:** Hệ thống yêu cầu AI phải "tóm tắt tài liệu môn học". Kiến trúc Pipe-and-Filter cho phép tái sử dụng các Filter (ví dụ: module "Tách văn bản" có thể dùng lại cho tính năng tìm kiếm sau này).
 2. **Ràng buộc xử lý bất đồng bộ (từ Câu 2):** Khi upload file, ta có thể đẩy file đó vào một Pipeline chạy ngầm. Giảng viên không cần chờ, khi nào luồng Pipe-and-Filter chạy xong, nó sẽ bắn ra một Event (thông báo) cho giảng viên.
- **Phương án bị loại: Batch Sequential (Xử lý lô tuần tự).**
 - *Lý do loại:* Giảng viên cần kết quả tóm tắt tài liệu (hoặc ít nhất là file được xử lý ngay) trong thời gian ngắn để chuẩn bị bài giảng. Nếu dùng Batch, hệ thống sẽ gom tất cả PDF của mọi giảng viên lại và đợi đến khuya mới xử lý một thể, điều này gây trải nghiệm người dùng rất tệ và không đáp ứng được tính tức thời của thao tác upload.

2. Phân tích Nhu cầu B: Chạy báo cáo thống kê lúc 1h sáng

Quyết định: Nhu cầu B sinh ra là để dành cho phong cách **Batch Sequential (Xử lý lô tuần tự)**.

- **Giải thích vì sao chọn:** Đây là bài toán xử lý một khối lượng dữ liệu khổng lồ (log truy cập của cả trường trong 24h) không yêu cầu thời gian thực. Các bước phải diễn ra tuần tự: Phải tổng hợp XONG toàn bộ log thì mới tính thống kê đúng được; tính thống kê XONG thì mới phát hiện được lớp có nguy cơ. Một bước phải hoàn thành 100% (có "Wait barrier") thì bước sau mới bắt đầu.
- **Gắn với 2 ràng buộc cụ thể:**

1. **Ràng buộc 5.000 CCU ban ngày:** Ban ngày hệ thống phải gồng gánh lượng truy cập khổng lồ. Việc quét toàn bộ DB để tính toán log học tập sẽ làm sập máy chủ. Đưa tác vụ này thành Batch chạy lúc 1h sáng sẽ tận dụng tài nguyên rảnh rỗi, bảo vệ hiệu năng hệ thống chính.
2. **Ràng buộc hệ thống Legacy 2018:** Đề bài đã cho biết trường có truyền thống đồng bộ CSV vào ban đêm. Việc tạo báo cáo bằng Batch Sequential lúc 1h sáng có thể dễ dàng ghép nối để xuất file CSV gửi luôn cho hệ thống cũ.

- **Phương án bị loại: Pipe-and-Filter.**

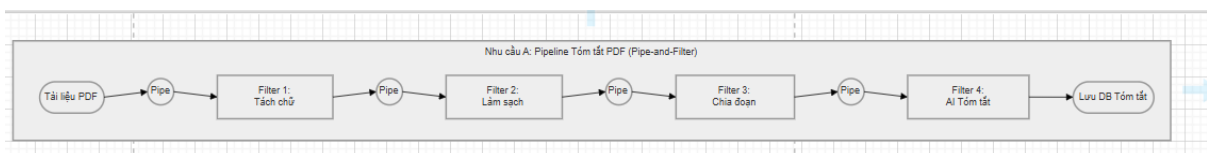
- *Lý do loại:* Pipe-and-Filter xử lý luồng dữ liệu liên tục. Nếu log cứ sinh ra là chảy qua Filter để tính toán lại báo cáo ngày thì hệ thống sẽ phải liên tục cập nhật/tính toán lại những con số thống kê chưa hoàn chỉnh, gây lãng phí CPU cực lớn mà kết quả phòng đào tạo xem lúc 3h chiều cũng chưa phải là kết quả chốt của ngày.

3. Giải thích sự khác biệt cốt lõi trong ngữ cảnh đề bài

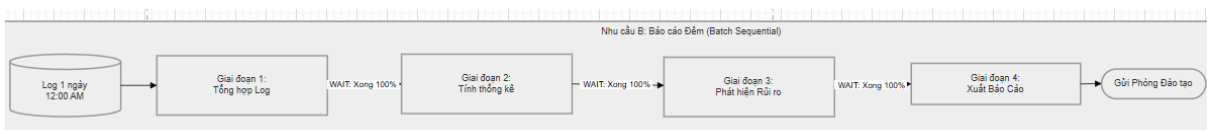
Mặc dù cả hai đều xử lý dữ liệu qua nhiều bước, nhưng sự khác biệt nằm ở **dòng chảy thời gian và trạng thái chờ**:

- **Trong Nhu cầu A (Pipe-and-Filter):** Dữ liệu chảy *liên tục* (Continuous). Trang 1 của file PDF có thể đang được "Làm sạch" ở Filter 2, trong khi trang 2 mới đang được "Tách văn bản" ở Filter 1. Các Filter chạy song song và độc lập. Nó giải quyết bài toán: "*Xử lý ngay món đồ này qua một dây chuyền*".
- **Trong Nhu cầu B (Batch Sequential):** Dữ liệu bị *chặn lại* (Discrete). Hệ thống gom đủ 100% dữ liệu log của ngày hôm đó, ném vào Giai đoạn 1. Phải đợi Giai đoạn 1 xử lý xong 100% đồng log đó thì mới mở cổng cho dữ liệu chạy sang Giai đoạn 2. Nó giải quyết bài toán: "*Gom đủ một mẻ lớn rồi xử lý từng công đoạn thật chính xác trong lúc vắng người*".

Sơ đồ chọn nhu cầu A:



Sơ đồ cho nhu cầu B:



Câu 5 :

1. Đây là loại vấn đề kiến trúc gì?

Đây là bài toán **Xung đột đồng bộ dữ liệu phân tán (Distributed Data Synchronization Conflict)**. Cụ thể hơn, nó thuộc dạng xung đột do **độ trễ trạng thái (State Staleness)**.

- Khi mất mạng, Client (app điện thoại) bị cô lập và tạo ra một "nhánh" trạng thái dữ liệu (điểm danh dựa trên danh sách lớp cũ).
- Trong khi đó, Server (nhờ sự can thiệp của phòng đào tạo) đã tiến lên một trạng thái mới (sinh viên đã chuyển lớp).
- Khi mạng có lại, hai nhánh này gộp (merge) vào nhau sinh ra xung đột logic nghiệp vụ.

2. Đề xuất cách xử lý ở mức kiến trúc (Kèm ràng buộc & Phương án loại)

Quyết định kiến trúc: Bổ sung một **Conflict Resolution Engine (Bộ giải quyết xung đột)** vào tầng Business Logic của Server, kết hợp với phong cách **Event-driven** để xử lý hậu quả.

- **Nguyên tắc đồng bộ (Data & Sync Principle):** Sử dụng cơ chế *Server-Authoritative* (Server là chân lý) kết hợp *Event-based Syncing*. Khi điện thoại đồng bộ lên, nó không gửi lệnh "Cập nhật sinh viên A có mặt", mà gửi một sự kiện chứa ngữ cảnh: "*Lúc 9h00, Giảng viên X điểm danh sinh viên A có mặt, dựa trên danh sách lớp phiên bản V1*". Khi Server nhận dữ liệu, Conflict Resolution Engine sẽ so sánh: Nếu sinh viên A không còn thuộc lớp đó nữa (danh sách đã là V2), Server sẽ **từ chối** bản ghi điểm danh của riêng sinh viên A (vẫn nhận của các sinh viên khác), lưu log, và phát ra một Event để báo lại cho Giảng viên.
- **Gắn với 2 ràng buộc trong đề:**
 1. **Wi-Fi chậm chờn:** Bắt buộc hệ thống phải chấp nhận độ trễ dữ liệu từ Client gửi lên.
 2. **Bảo mật và Phân quyền:** Phòng đào tạo nắm quyền cao nhất về danh sách lớp. Dữ liệu điểm danh của giảng viên (dù đúng thời điểm đó) không được phép "vượt mặt" hoặc làm hỏng cấu trúc danh sách do phòng đào tạo quản lý.
- **Phương án bị loại: Last-Write-Wins (LWW - Lần ghi cuối luôn thắng).**
 - *Lý do loại:* LWW là cách dễ nhất (thắng nào đồng bộ lên sau cùng thì đè data thắng trước). Nếu áp dụng, app của giảng viên (đồng bộ lúc có mạng) sẽ ép Server ghi nhận sinh viên A đi học ở lớp cũ, phá hỏng quyết định chuyển lớp của phòng đào tạo trước đó. Dữ liệu sẽ bị sai bét.

3. Hậu quả nếu xử lý sai mức kiến trúc

- **Rác dữ liệu (Data Anomaly) & Crash hệ thống:** Dữ liệu điểm danh bị lưu vào một lớp học mà sinh viên không còn thuộc về. Lúc này, nếu Database có ràng buộc khóa ngoại (Foreign Key Constraint) chặt chẽ, việc map dữ liệu sẽ văng lỗi (Exception), làm sập luôn luồng đồng bộ của toàn bộ lớp học đó.
- **Sai lệch nghiệp vụ nghiêm trọng:** Sinh viên A đi học ở lớp mới nhưng bị đánh vắng vì giảng viên lớp mới chưa có tên trong danh sách, trong khi ở lớp cũ thì lại có lịch sử điểm danh. Đến cuối kỳ xét điều kiện cấm thi sẽ rối tung lên, không biết lấy data nào làm chuẩn.